

STAFF ENGINEER PREPARATION GUIDE

Designing Data-Intensive Applications

The Big Ideas Behind Reliable, Scalable, and Maintainable Systems

Simplified summary of Martin Kleppmann's book —
plain language, real-world examples, and diagrams

Covers all 12 chapters • Part I: Foundations • Part II: Distributed Data • Part III: Derived Data

Contents

Part I — Foundations of Data Systems

2. Reliable, Scalable, and Maintainable Applications
3. Data Models and Query Languages
4. Storage and Retrieval
5. Encoding and Evolution

Part II — Distributed Data

5. Replication
6. Partitioning (Sharding)
7. Transactions
8. The Trouble with Distributed Systems
9. Consistency and Consensus

Part III — Derived Data

10. Batch Processing
11. Stream Processing
12. The Future of Data Systems

Appendix

One-Page Cheat Sheet for Interviews & Design Reviews

How to use this guide

Each chapter is summarized in simple language with three recurring blocks:

- **Example** — a concrete, real-world scenario that makes the idea click.
- **Key idea** — the one thing to remember from a section.
- **Staff engineer lens** — how the topic shows up in design reviews, trade-off discussions, and interviews.

The diagrams are intentionally simple. If you can redraw them on a whiteboard from memory, you understand the concept well enough to defend it in a design discussion.

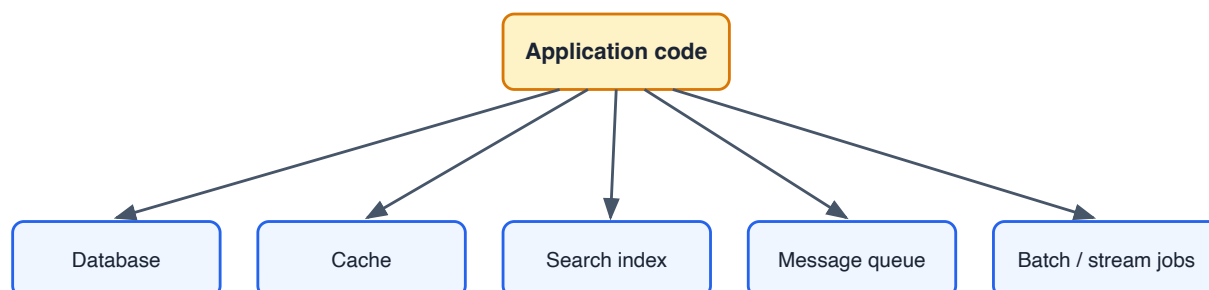
PART I

Foundations of Data Systems

The ideas that apply to every system, whether it runs on one machine or a thousand: what "reliable, scalable, maintainable" actually mean, how to model data, how databases store and find it, and how to change data formats without breaking things.

Chapter 1 — Reliable, Scalable, and Maintainable Applications

Modern applications are rarely limited by CPU. They are limited by **data** — its volume, its complexity, and how fast it changes. Nobody builds a data system from scratch; we compose building blocks: databases, caches, search indexes, message queues, batch/stream processors. Your job as an engineer is to combine them so that the whole system has three properties: **reliability, scalability, and maintainability**.



You are the architect of a **composite data system** — the API hides many moving parts.

Figure 1-1. Every real application is a composite of specialized data tools stitched together by application code.

Reliability — "works correctly even when things go wrong"

A **fault** is one component misbehaving; a **failure** is the whole system stopping to serve users. Good systems are **fault-tolerant**: faults happen, failures don't. There are three fault families:

- **Hardware faults** — disks die (a 10,000-disk cluster loses ~1 disk per day), RAM corrupts, power fails. Cure: redundancy and, at scale, software tolerance (replicas across machines).
- **Software errors** — bugs that hit *all* nodes at once (e.g., the leap-second bug that crashed many Linux servers simultaneously). Correlated, so redundancy alone doesn't help. Cure: careful design, testing, process isolation, monitoring, self-checks.
- **Human errors** — the #1 cause of outages (config mistakes beat hardware faults). Cure: good abstractions, sandboxes, easy rollback, gradual rollouts, telemetry.

Example: Netflix deliberately kills random production servers with Chaos Monkey. If killing a machine at noon on a Tuesday causes an outage, they'd rather learn it on their schedule than at 3 a.m. Deliberately inducing faults proves fault tolerance actually works.

Scalability — "keeps working as load grows"

Scalability is not a yes/no label. The right question is: "If load grows in a particular way, what are our options for coping?" First describe **load parameters** (requests/sec, read/write ratio, fan-out, cache hit rate), then measure **performance**.

Example — Twitter's fan-out problem: Posting a tweet is easy (~5K/sec peak). The hard part is the home timeline: each user follows many people. Two designs:

1. **Pull (read-time join):** store tweets once; when a user opens the app, query tweets from everyone they follow. Cheap writes, expensive reads.
2. **Push (write-time fan-out):** when someone tweets, insert it into the cached timeline of each follower. Expensive writes (a celebrity with 30M followers = 30M writes), instant reads.

Twitter ended up with a **hybrid**: push for normal users, pull for celebrities. Lesson: the distribution of your load (followers per user) drives architecture, not the average.

Percentiles, not averages

Response time is a **distribution**. The average hides pain; use percentiles. p50 = the typical user; p99 = 1 in 100 requests is slower than this. Tail latency matters because your slowest requests often belong to your most valuable users (they have the most data), and because one user request may fan out to many backend calls — the slowest backend call gates the whole response (**tail latency amplification**).

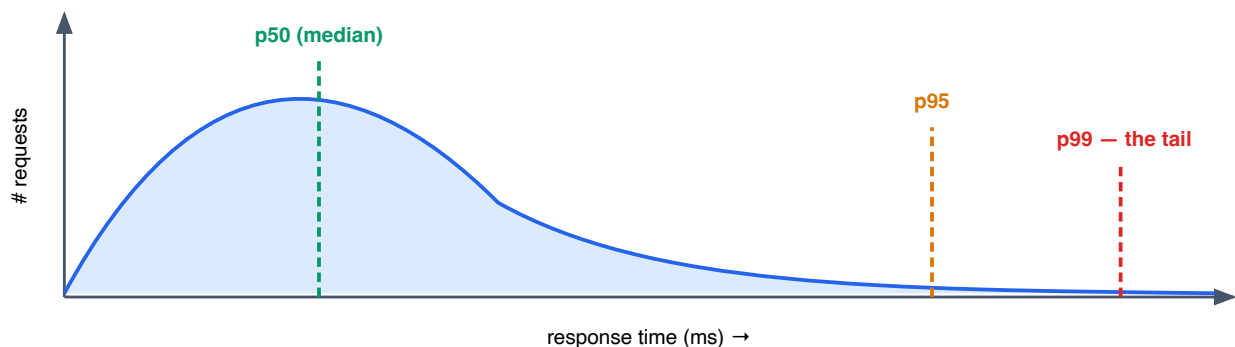


Figure 1-2. Most requests are fast, but the long tail (p95/p99) defines the experience of real users. SLOs/SLAs are written in percentiles.

Scaling up (bigger machine) vs **scaling out** (more machines): single machines are simpler; distribute when cost or availability forces you to. Stateless services are easy to scale out; **stateful data is the hard part** — which is why the rest of the book exists. There is no one-size-fits-all architecture: a system for 100,000 small requests/sec looks nothing like one for 3 huge requests/min, even at the same throughput.

Maintainability — "future engineers can work on it"

- **Operability** — make life easy for ops: good monitoring, docs, sane defaults, self-healing, predictable behavior.
- **Simplicity** — fight accidental complexity with **abstraction** (SQL is an abstraction over B-trees; TCP over unreliable packets). A "big ball of mud" slows everyone down and breeds bugs.
- **Evolvability** — make change easy. Requirements will change; design so the system can be reshaped (this connects to schema evolution in Chapter 4).

Key idea: Reliability = tolerate faults (hardware, software, human). Scalability = describe load, measure percentiles, have a plan for 10× growth. Maintainability = operability + simplicity + evolvability. Every later chapter is a tool for buying these three properties.

Staff engineer lens: In design reviews, always ask: (1) What are the load parameters and which one will grow first? (2) What is the p99 target and what happens during fan-out? (3) What is the blast radius of a bad deploy or a fat-fingered config? Framing discussions with fault/failure, load parameters, and percentiles signals senior judgment more than naming specific technologies.

Chapter 2 — Data Models and Query Languages

Data models are the most important decision in software: they shape not just the code, but *how you think about the problem*. Each layer hides the one below it: objects → JSON/tables → bytes on disk. This chapter compares the three big general-purpose models: **relational**, **document**, and **graph**.

Relational vs Document

The relational model (SQL, 1970) organizes data into tables (relations) and rows (tuples). It won for decades because it generalizes well and hides implementation details behind a declarative language. NoSQL document databases (MongoDB, etc.) emerged around 2010, driven by: need for bigger scale/write throughput, desire for open-source, specialized queries, and frustration with rigid schemas.

The **object-relational mismatch** ("impedance mismatch"): your code has nested objects; SQL has flat tables — so you need ORMs or joins to translate. A résumé (one user, many jobs, many schools) is a **tree**, and JSON stores a tree naturally, with better **locality** (one read fetches the whole profile).

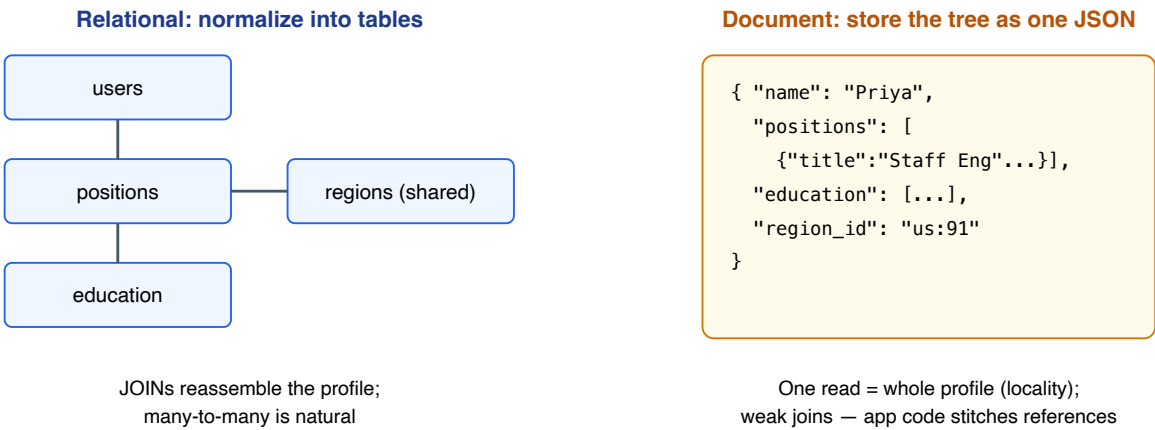


Figure 2-1. The same LinkedIn-style profile in the two models. One-to-many trees fit documents; many-to-many relationships fit relations.

When data gets interconnected, documents hurt. If "region" or "company" become entities of their own (many-to-one), or users can reference each other (many-to-many), documents force you to either duplicate data (denormalize) or emulate joins in application code — which is slower and more code. This is literally history repeating: IBM's 1970s hierarchical IMS model had the same limits, and the relational model won that argument.

Which to choose?

Prefer document model when...	Prefer relational when...
Data is tree-shaped, self-contained, loaded all at once (profile, event payload, product page)	Many-to-many relationships and joins are common
Schema changes often or records vary in shape ("schema-on-read")	You want the database to enforce structure ("schema-on-write")
Locality: read the whole document in one go	You typically read small parts of large records (docs must be rewritten wholesale on update)

The models are **converging**: PostgreSQL has JSON columns; some document stores added joins. Pick per use case, not per religion.

Key idea: Schema-on-read vs schema-on-write is like dynamic vs static typing — neither is "better"; the shape and interconnectedness of your data decides. One-to-many tree → document. Many-to-many web → relational or graph.

Declarative beats imperative

SQL is **declarative**: you say *what* you want, the query optimizer decides *how*. That frees the database to change indexes, parallelize, and reorder without breaking your query. Imperative code (loop over rows) locks in one execution strategy. Same idea as CSS vs manually walking the DOM. MapReduce querying sits in between — a programming model where you supply pure `map` and `reduce` functions and the framework handles distribution.

Graph data models

When **anything can relate to anything** (social networks, road networks, fraud rings, recommendation engines), graphs shine. A **property graph** = vertices + directed edges, each with key-value properties. Query languages like **Cypher** let you say "find people who emigrated from the US to Europe" in a few lines, where SQL needs painful recursive CTEs — because in a graph you can traverse a *variable number of edges* naturally.

Example: Fraud detection at a payments company: accounts, devices, cards, and addresses are vertices; "used-by" and "shipped-to" are edges. "Find accounts within 3 hops of a known-fraudulent card" is a one-line graph traversal but an unbounded self-join nightmare in SQL.

Staff engineer lens: The interview-ready framing: "I'd model the profile as a document for read locality, keep orders relational because they join to inventory and payments, and use a graph store only if multi-hop traversals are a core access pattern." Choosing *per access pattern*, and knowing that denormalization trades write complexity for read speed, is the signal.

Chapter 3 — Storage and Retrieval

A database does two things: **store data when you give it**, **return data when you ask**. Knowing roughly how it does that lets you pick the right engine and tune it. There are two dominant families of storage engines — **log-structured (LSM-trees)** and **page-oriented (B-trees)** — plus a separate world for analytics (**column stores**).

The simplest database, and why indexes exist

The world's simplest database is two shell functions: append `key, value` to a file to write; grep the file for the last occurrence to read. Appending is blazing fast (sequential write), but reading is $O(n)$. An **index** is extra metadata that speeds up reads — at the cost of slowing down writes (every write must also update the index).

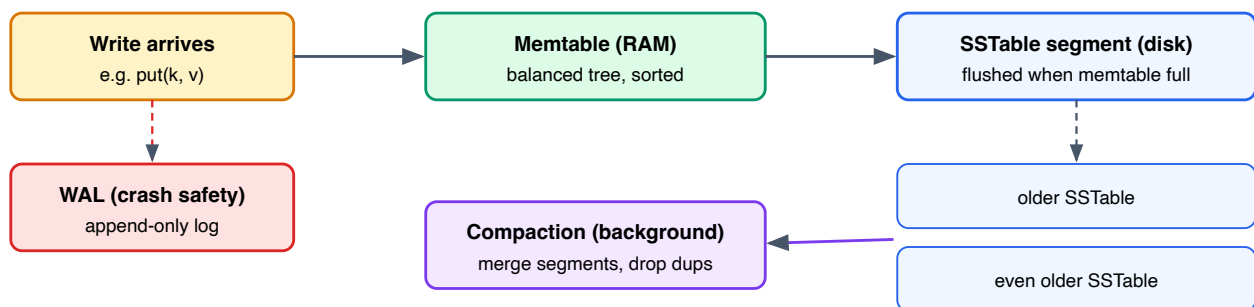
This is the fundamental trade-off of storage: indexes make reads faster and writes slower. That's why databases don't index everything by default; you choose.

Hash indexes

Keep an in-memory hash map: `key` → byte offset in the append-only file (this is Bitcask/Riak). To avoid running out of disk, break the log into **segments** and periodically **compact** them (keep only the latest value per key). Great for lots of updates to a small set of keys (e.g., view counters). Limits: all keys must fit in RAM, and **range queries are impossible** (keys are scattered randomly).

SSTables and LSM-trees (Cassandra, RocksDB, LevelDB, HBase)

Improvement: keep each segment file **sorted by key** — a *Sorted String Table* (SSTable). Now merging segments is a simple mergesort, you only need a sparse index in memory, and range scans work. How do you write sorted files when writes arrive in any order?



Read path: memtable → newest SSTable → older ones. Bloom filters skip segments that can't contain the key.

Figure 3-1. LSM-tree write path: writes hit RAM, get flushed as sorted runs, and compaction merges runs in the background.

B-trees (PostgreSQL, MySQL/InnoDB, Oracle — the default since 1970s)

B-trees break the database into fixed-size **pages** (~4 KB) forming a tree. Each page holds sorted keys and child pointers; a lookup walks root → leaf in 3–4 hops (a 4-level tree of 4 KB pages with 500-way branching covers 250 TB). Writes **overwrite pages in place** — unlike LSM's append-only style. Crash safety comes from a **write-ahead log (WAL)**: every change is appended to the WAL before touching the page.

B-tree vs LSM-tree — the trade-off table

LSM-tree (log-structured)		B-tree (page-oriented)
Writes	Faster: sequential appends, higher throughput, less write amplification, better compression	Slower: random page writes; every page rewrite ≥ 4 KB even for 1 byte
Reads	Slower: may check memtable + several SSTables	Faster & predictable: one tree walk; each key lives in exactly one place
Background cost	Compaction can steal disk bandwidth; badly tuned compaction falls behind at high write rates → p99 spikes	No compaction, but fragmentation wastes space
Transactions	Harder (same key in many places)	Easier: lock ranges directly in the tree — why classic RDBMSs use them

Key idea: LSM = optimized for write-heavy workloads; B-tree = predictable read latency and simpler transactional locking. When someone proposes Cassandra vs Postgres, this table is usually the real conversation.

Other indexes in one breath

- **Secondary index:** same structures, but values aren't unique — the index maps a value to a list of matching row IDs.
- **Clustered vs non-clustered:** store the actual row inside the index (clustered, like MySQL InnoDB primary key) vs store a pointer into a heap file.
- **Covering index:** stores some columns inside the index so queries can be answered from the index alone.
- **Multi-column / concatenated index:** like a phone book (lastname, firstname) — great for queries matching the prefix.
- **Full-text / fuzzy:** Lucene-style term dictionaries with tolerance for typos.
- **In-memory databases** (Redis, VoltDB): faster not because "RAM vs disk reads" (OS caches hot data anyway) but because they skip the overhead of *encoding data structures for disk*.

OLTP vs OLAP, and column stores

OLTP (transaction processing): many small reads/writes of individual records by key — user-facing. **OLAP** (analytics): few huge queries scanning millions of rows, aggregating a few columns — analyst-facing. Running both on one database causes analysts' scans to trash the OLTP cache, so companies extract data (ETL) into a **data warehouse**, typically modeled as a **star schema**: one giant *fact table* of events, surrounded by *dimension tables* (who/what/where/when).

Row storage (OLTP)

id=1		2026-07-01		sku=42		\$9.99
id=2		2026-07-01		sku=17		\$4.50
id=3		2026-07-02		sku=42		\$9.99

Whole rows together —
good for "fetch order #2"

Column storage (OLAP)

date: 07-01, 07-01, 07-02 ...
sku: 42, 17, 42 ...
price: 9.99, 4.50, 9.99 ...

Each column in its own file — scan only
what the query needs; compresses superbly

SELECT SUM(price) WHERE date='07-01' touches 2 small files instead of every row.

Figure 3-2. Row vs column layout. Column stores (Redshift, BigQuery, Parquet) win analytics because queries read few columns of many rows.

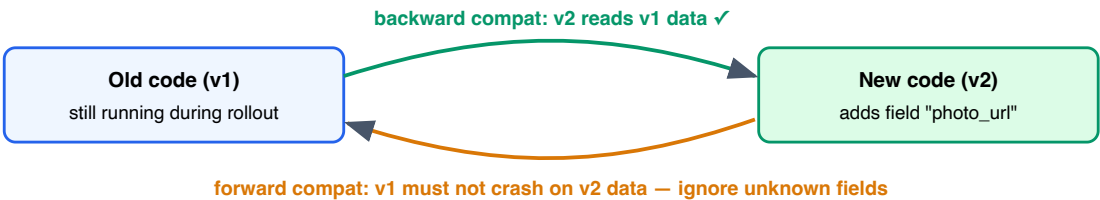
Column values compress extremely well (bitmap + run-length encoding, because values repeat), which also makes better use of CPU caches. Writes are handled LSM-style: buffer in memory, rewrite column files in bulk. **Materialized views / data cubes** pre-compute common aggregates as a cache of query results.

Staff engineer lens: Two evergreen review questions: "Is this workload OLTP or OLAP — and are we accidentally running analytics on the OLTP primary?" and "Is the write path append-heavy enough to justify an LSM engine, or do we value predictable read p99 and transactional locking (B-tree)?" Also: every index you add is a tax on every write — ask which queries actually justify each index.

Chapter 4 — Encoding and Evolution

Code changes constantly, and data outlives code. During a rolling deploy, **old and new code, and old and new data formats, coexist at the same time**. So every data format needs two properties:

- **Backward compatibility:** new code can read data written by old code (usually easy).
- **Forward compatibility:** *old* code can read data written by *new* code (harder — old code must ignore fields it doesn't know about).



Rolling deploys, blue/green, and long-lived DB rows all force both directions to work.

Figure 4-1. During any rolling upgrade both versions run simultaneously — compatibility must hold in both directions.

Encoding formats compared

Format	How schema works	Watch out for
Language-native serialization (Java Serializable, pickle)	None	Don't use for storage/APIs: locks you into one language, security holes, no versioning. Transient use only.
JSON / XML / CSV	Optional, external	Human-readable, universal. But: numbers are ambiguous (JS mangles integers $> 2^{53}$ — Twitter ships tweet IDs as both number <i>and</i> string), no binary strings (Base64 hack), verbose.
Thrift / Protocol Buffers	Required schema with numbered field tags	Compact binary. Never reuse or change a tag number. New fields must be optional or have defaults.
Avro	Two schemas: writer's and reader's , resolved at read time; no tags in data	Smallest encoding. Friendliest to <i>dynamically generated</i> schemas (e.g., dump every DB table). Reader needs access to writer's schema (file header or schema registry).

Example — why field tags matter: In protobuf, `name = 1, user_id = 2`. The wire format only contains "field 1: Priya", "field 2: 42". You can freely rename `user_id` to `account_id` — the wire never sees names. But if you delete field 2 and later reuse tag 2 for something else, old data suddenly parses as garbage. Tag numbers are forever.

Key idea: Schemas are not the enemy of flexibility — schema-driven binary formats (Protobuf/Avro) give you *documented, machine-checked evolution rules*, better than hoping every JSON producer and consumer agrees. Compatibility must be maintained across **rolling upgrades** and across **data at rest that outlives code by years**.

Three ways data flows between processes

- **Via databases:** "sending a message to your future self." Beware: an old-code process that reads a row with a new field and writes it back may silently **drop the new field**. Migrations of huge tables are expensive, so most databases allow adding nullable columns — schema evolution again.
- **Via services (REST/RPC):** servers get updated before clients (usually) — so servers need backward-compatible *requests*, clients need forward-compatible *responses*. RPC's core flaw: it pretends a network call is a local function call, but network calls can time out, duplicate, and reorder — design APIs expecting that. Public APIs are the hardest: you don't control your clients, ever.
- **Via message brokers (Kafka, RabbitMQ):** decouples sender from recipient — broker buffers if the consumer is down, one message can fan out to many consumers, sender doesn't wait for a reply. Same encoding rules apply.

Staff engineer lens: The staff-level habit is asking "what happens mid-rollout?" for every schema and API change: Can v1 read what v2 wrote? Does any old consumer rewrite records and silently strip new fields? Is there a schema registry / contract test enforcing this, or just convention? Systems where every change requires a lockstep deploy are systems that can't evolve — evolvability (Ch. 1) is bought here.

PART II

Distributed Data

What changes when data lives on multiple machines: copying it (replication), splitting it (partitioning), keeping multi-step operations correct (transactions), surviving unreliable networks and clocks, and getting machines to agree (consensus).

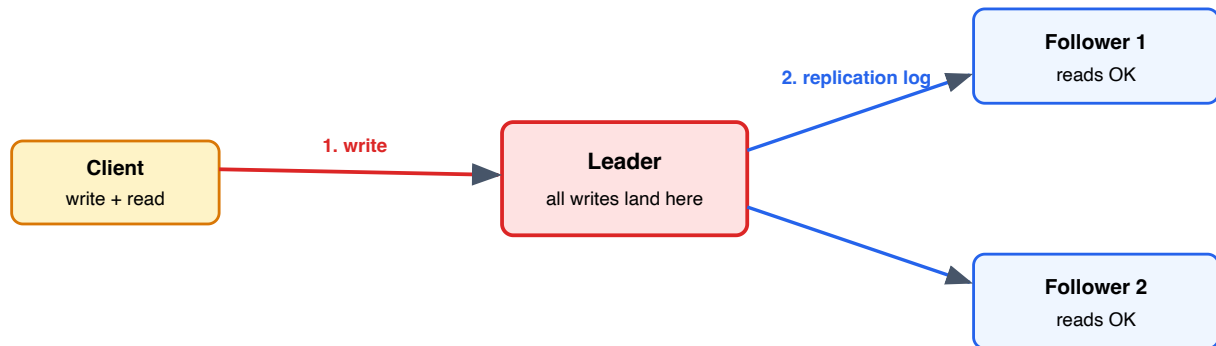
Why distribute at all? **Scalability** (load too big for one machine), **availability** (survive machine/datacenter loss), and **latency** (data near users worldwide).

Chapter 5 — Replication

Replication = keeping a copy of the same data on multiple machines. Easy if data never changed; the whole difficulty is **handling changes**. Three architectures: **single-leader**, **multi-leader**, **leaderless**.

Single-leader (leader/follower) — the default

All writes go to one node (the **leader**); it streams a change log to **followers**; reads can go to any node. This is how PostgreSQL, MySQL, MongoDB, and Kafka replication work.



Followers apply changes in the same order the leader processed them.

Figure 5-1. Leader-based replication: one write path, many read paths.

Sync vs async: synchronous replication guarantees the follower has the write (durable) but stalls all writes if that follower is slow/dead. Fully synchronous is impractical, so real systems are async or "semi-sync" (one sync follower). **Async means: if the leader dies, recent writes may be lost.**

Failover is where the bodies are buried. When the leader dies you must: detect it (timeout — but how long?), elect a new leader, repoint clients. Failure modes: the old leader's unreplicated writes get discarded (GitHub once reused MySQL primary keys this way, leaking private data via a Redis cache that had seen the old keys); **split brain** (two nodes both think they're leader → data corruption); too-short timeouts cause needless failovers during load spikes.

Replication lag and read-your-writes

Async followers lag behind the leader — usually milliseconds, sometimes minutes under load. This creates real, user-visible anomalies (this is "eventual consistency" being annoying):

Anomaly	What the user sees	Guarantee that fixes it
You post a comment, refresh, it's gone	Read hit a follower that hasn't caught up with your own write	Read-your-writes: read your own data from the leader (or a caught-up replica)
Comment appears, then disappears on next refresh	Second read hit a more-lagged replica — time went backwards	Monotonic reads: pin each user to the same replica
You see an answer before the question it replies to	Two causally-related writes replicated out of order	Consistent prefix reads: causally related writes land in the same order everywhere

Example: User updates their profile photo (write → leader), then immediately views their profile (read → lagging follower) and sees the old photo — files a bug: "my save didn't work." Standard fix: for one minute after a user writes, serve that user's profile reads from the leader.

Multi-leader replication

Multiple nodes accept writes; each forwards changes to the others. Use cases: **multi-datacenter** (a leader per DC — writes stay local, survive DC outage), **offline clients** (your phone's calendar is a "leader" that syncs later), **collaborative editing** (Google Docs). The price: **write conflicts**. Two users update the same record on different leaders; both succeed locally; the conflict is detected only later, asynchronously.

Conflict handling options: avoid conflicts (route each record's writes to one home leader — recommended), **last-write-wins** (pick a timestamp winner — silently *discards data*), record the conflict and let application code or the user merge (like git). CRDTs are emerging data structures that merge automatically.

Leaderless replication (Dynamo-style: Cassandra, Riak)

No leader at all: the client writes to **all N replicas in parallel** and considers the write successful after **w** acknowledge; reads query all and take the newest of **r** responses. If $w + r > n$, read and write sets overlap, so a read should see the latest write (a **quorum**).

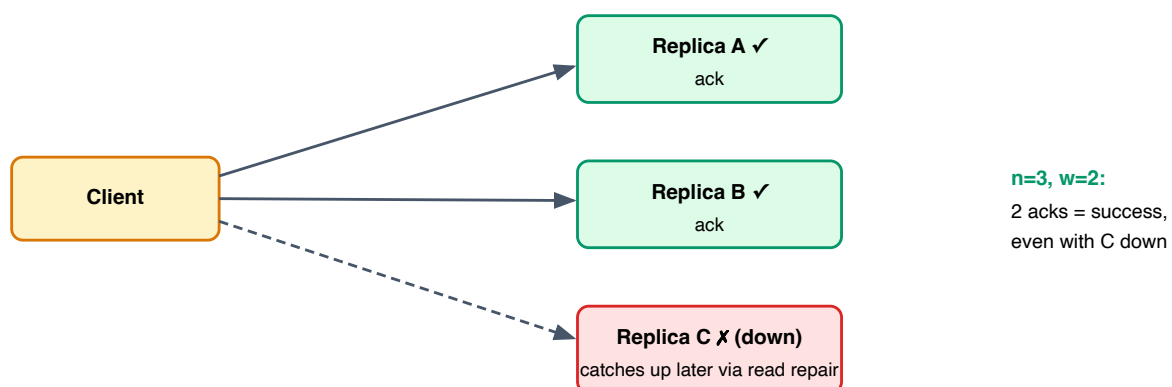


Figure 5-2. Leaderless write with $n=3$, $w=2$: no failover needed; stale replicas are fixed by read repair and anti-entropy.

Quorums are **not** as strong as they look: sloppy quorums (writes land on stand-in nodes during outages), concurrent writes, and partial write failures all break the "read sees latest write" promise. Concurrency is handled with **version vectors**, which detect that two writes were concurrent so *siblings* can be merged by the app — or crudely by last-write-wins.

Key idea: Single-leader = simple, ordered, but failover is scary. Multi-leader = multi-DC and offline-first, but you own conflict resolution. Leaderless = high availability with no failover machinery, but weaker consistency semantics. Every replication scheme trades consistency for availability/latency somewhere — know where.

Staff engineer lens: Interview gold: walk through what a user experiences under replication lag, then name the exact guarantee needed (read-your-writes / monotonic reads) rather than reaching for "strong consistency everywhere" — that shows cost awareness. Also always ask about failover runbooks: who promotes the new leader, and what happens to writes the dead leader hadn't replicated?

Chapter 6 — Partitioning (Sharding)

Replication copies *all* the data to several nodes; **partitioning splits** the data so each node holds only a piece. You partition when the dataset or write throughput is too big for one machine. Partitioning and replication are combined: each partition has replicas on several nodes.

How to split: key range vs hash

	Key-range partitioning	Hash partitioning
How	Assign contiguous ranges (A–C → node 1, D–F → node 2...), like encyclopedia volumes	Partition by hash(key) — spreads keys evenly
Range scans	Efficient — adjacent keys sit together (great for time-series queries)	Lost — adjacent keys scatter across partitions
Hot spots	Danger: writing today's data with timestamp keys sends <i>all writes to one partition</i>	Even spread — but a single celebrity key can still be hot
Used by	HBase, RethinkDB, MongoDB (range mode)	Cassandra, DynamoDB, Voldemort

Example — hot spot fix: A sensor platform keyed writes by `timestamp` — every write hit the same partition ("today's"). Fix: prefix the key with sensor ID (`sensor_id`, `timestamp`) so writes spread across partitions, while per-sensor range queries stay fast. For a celebrity key (e.g., a viral post's counter), append a random 2-digit suffix to split one key across 100 partitions — accepting that reads must now gather all 100.

Secondary indexes across partitions

- **Local index (by document):** each partition indexes only its own rows. Writes are cheap (touch one partition), but a query on the indexed field must **scatter/gather across all partitions** — tail latency amplification.
- **Global index (by term):** the index itself is partitioned by the indexed value ("all red cars" lives in one place). Reads hit one partition, but one write must update indexes on *multiple* partitions — usually async.

Rebalancing — moving data when nodes change

Never use hash mod N: adding one node changes the assignment of almost every key — a full reshuffle. Instead:

- **Fixed number of partitions** (e.g., 1,000 partitions across 10 nodes): a new node *steals whole partitions* from others; only those move. Simple, most common (Riak, Elasticsearch, Kafka).
- **Dynamic partitioning:** partitions split when too big, merge when small (HBase, MongoDB) — adapts to data volume.

Keep a **human in the loop** for rebalancing: an auto-rebalancer plus an overloaded, slow-to-respond node is a recipe for cascading failure (the system declares the slow node dead and moves data *onto already-busy nodes*).

Request routing

How does a client find the right partition? Three options: any node forwards (gossip, Cassandra), a routing tier (ZooKeeper-backed, Kafka/HBase), or partition-aware clients. The underlying problem — everyone agreeing on who owns what — is consensus, previewing Chapter 9.

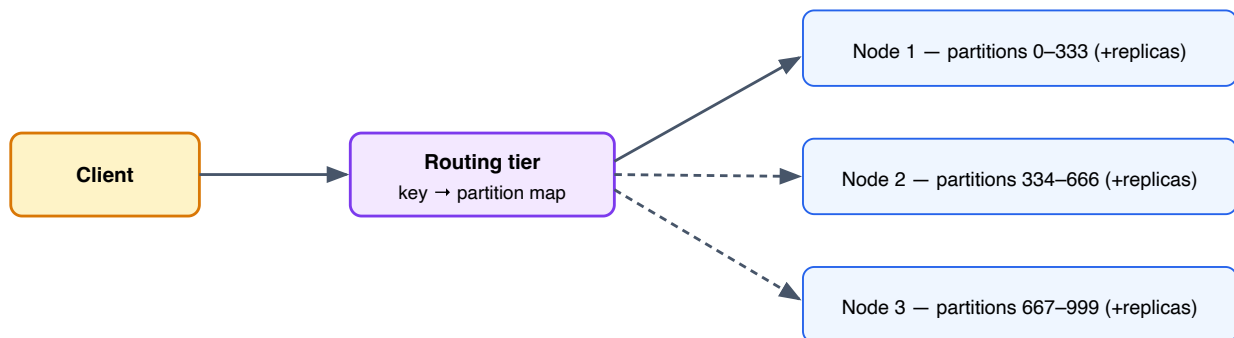


Figure 6-1. 1,000 fixed partitions spread over 3 nodes; a routing tier maps $\text{hash}(\text{key}) \rightarrow \text{partition} \rightarrow \text{node}$. Adding node 4 just moves some partitions.

Key idea: Partitioning goal = spread data *and* load evenly. Hash for even spread, ranges for scans — and design keys so neither creates a hot spot. Rebalance by moving whole partitions, never by re-hashing everything.

Staff engineer lens: The question that separates levels: "what's your partition key, and what does the worst query fan out to?" A design whose common query is a scatter/gather across 100 partitions inherits the p99 of the slowest partition. Also probe skew: one hot tenant/celebrity can sink a 'perfectly balanced' hash scheme.

Chapter 7 — Transactions

Lots of things can go wrong mid-operation: crashes, network failures, concurrent writers, partial writes. A **transaction** groups several reads/writes into one all-or-nothing unit so your application can ignore that whole class of problems. Transactions are not "slow and old-fashioned" nor sacred — they're a **trade-off you should choose deliberately**.

ACID, honestly defined

- **Atomicity** — really *abortability*: if anything fails halfway, all the transaction's writes are discarded, and it's safe to retry. (Not about concurrency.)
- **Consistency** — your app's invariants stay true (credits = debits). This is the *application's* job, not the database's; it's in the acronym mostly for the vowel.
- **Isolation** — concurrent transactions can't step on each other; ideally as if run one at a time ("serializable"). In practice, most databases default to something *weaker*.
- **Durability** — once committed, data survives crashes (WAL, replication). Perfect durability doesn't exist — only risk reduction.

The zoo of concurrency bugs, and which isolation level kills which

Race condition	Plain-language example	Fixed by
Dirty read	You see another transaction's half-finished, uncommitted writes	Read committed
Dirty write	You overwrite uncommitted data — car sale: Alice wins the listing, Bob gets the invoice	Read committed (row locks)
Read skew	You read account A before a transfer and account B after — money seems to vanish. Fatal for backups & analytics	Snapshot isolation (MVCC)
Lost update	Two counter increments read 42, both write 43 — one +1 vanishes	Atomic ops (<code>UPDATE ... SET v = v+1</code>), <code>SELECT FOR UPDATE</code> , or CAS
Write skew	Two doctors both check "≥2 on call" and both go off call — each write is fine alone, together they break the rule	Only serializable isolation
Phantom	Two users book the same meeting room: each checked "no booking exists", then both insert. The conflict is on a row that <i>didn't exist yet</i>	Serializable (or materialize the conflict as a lockable row)

Write skew: the on-call doctors problem

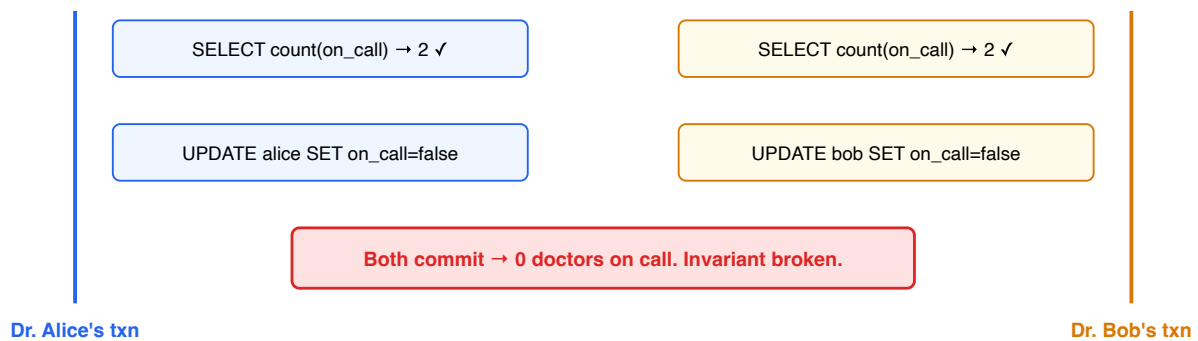


Figure 7-1. Write skew: each transaction reads a condition, then writes something that invalidates the other's premise. Snapshot isolation does not catch this.

How isolation is implemented

- **Read committed** (default in PostgreSQL, Oracle): row-level write locks + remember old value during writes so reads never see uncommitted data.
- **Snapshot isolation / MVCC**: each transaction reads from a **consistent snapshot** — the database keeps multiple versions of every row; "readers never block writers, writers never block readers." Perfect for long analytics queries and backups running alongside OLTP traffic.
- **Serializable** — three routes:
 1. **Actual serial execution** (Redis, VoltDB): one thread, no concurrency at all. Works if every transaction is tiny and the dataset fits in RAM; requires stored-procedure style (no waiting on the app mid-transaction).
 2. **Two-phase locking (2PL)**: readers block writers and vice versa; locks held until commit. Correct but poor throughput and unstable latency; deadlocks need detection.
 3. **Serializable Snapshot Isolation (SSI)**: optimistic — run on snapshots without blocking, track dependencies, and *abort at commit* any transaction that acted on a premise another transaction invalidated. Modern default answer (PostgreSQL `SERIALIZABLE`); performs well unless contention (abort rate) is high.

Key idea: "We use transactions" tells you nothing — *which isolation level?* Most databases default to read-committed or snapshot isolation, which permit lost updates, write skew, and phantoms. Any invariant of the form "check a condition, then write" (uniqueness, minimum balances, capacity limits) needs serializable isolation or explicit locking.

Example: A booking system enforced "room can't be double-booked" with `SELECT ... WHERE overlapping` then `INSERT` under snapshot isolation. Two customers booked the same slot in the same second — both `SELECT`s saw no conflict (phantom). Fixes: PostgreSQL exclusion constraint, `SERIALIZABLE`, or a lockable per-room-per-slot row.

Staff engineer lens: Audit any "check-then-act" logic in your codebase against the actual isolation level — this is one of the most common latent-bug classes in production systems. And when someone says "we'll enforce it in application code," ask what happens when two app servers run that code concurrently.

Chapter 8 — The Trouble with Distributed Systems

This chapter is deliberately pessimistic: on a single computer, operations either work or fail cleanly. In a distributed system, parts can fail **partially and nondeterministically** — and you may not even know whether something failed. Three untrustworthy pillars: **the network, clocks, and your own process's execution**.

Unreliable networks

Networks are asynchronous: when you send a request and hear nothing back, you cannot tell whether (a) the request was lost, (b) the remote node crashed, (c) the response was lost, or (d) everything worked and the reply is just slow. **The only tool is a timeout — and a timeout tells you nothing about what actually happened.** If the operation wasn't idempotent, retrying may do it twice.

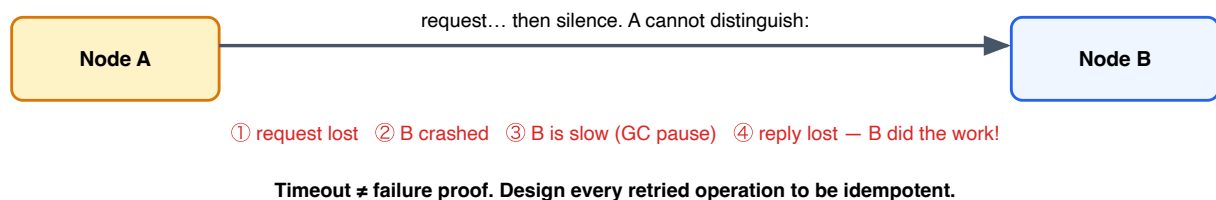


Figure 8-1. The fundamental uncertainty: no reply has four different explanations, and they require different responses.

Network faults are common even inside one datacenter (switch firmware, sharks biting cables, a misconfigured node absorbing traffic). Timeout tuning is a trade-off: short timeouts detect faults faster but mis-declare slow nodes dead — and shifting their load elsewhere during a spike can trigger **cascading failure**.

Unreliable clocks

- **Time-of-day clocks** (NTP-synced) **jump** — forward or backward — when corrected. Never use them to measure durations or order events.
- **Monotonic clocks** only move forward: use them for timeouts and latency measurements.
- Clock skew between machines is typically tens of milliseconds, and unbounded when NTP fails or VMs pause.

Deadly pattern — last-write-wins with wall clocks: if node A's clock is 30 ms ahead, A's *earlier* write can beat B's *later* write, silently dropping data with no error. Google Spanner's TrueTime handles this honestly: the clock API returns an *uncertainty interval*, and Spanner waits out the interval before committing so timestamp order matches real order — that's why it needs GPS/atomic clocks in every datacenter.

Process pauses

Your code can stop dead for seconds at any line: GC stop-the-world pauses, VM migration, laptop lid closes, thrashing swap. So even this is broken:

Example — the paused leaseholder: A node checks "I hold the lease, it's valid for 10 more seconds" and starts writing. A 15-second GC pause hits *between the check and the write*. The lease expires, another node takes over, and the zombie wakes up and writes anyway — corrupting data. You cannot fix this with more checking, because the pause can strike after any check.

The cure: majorities and fencing tokens

- **The truth is defined by the majority:** a node cannot trust its own judgment ("I'm alive! I hold the lock!"). Decisions like "is that node dead" are made by **quorum vote**, so no single confused node can wreak havoc.
- **Fencing tokens:** every time the lock service grants a lease it includes a monotonically increasing number. The storage system **rejects writes with an older token than it has already seen** — so the zombie's stale write bounces off. Checking on the resource side is essential; you can't trust the clients.



The resource enforces token order — stale leaseholders can't corrupt data no matter how confused they are.

Figure 8-2. Fencing tokens: the storage layer rejects the zombie's stale token. Safety without trusting clients or clocks.

Key idea: Assume: messages get lost, delayed, duplicated, reordered; clocks are wrong; processes pause arbitrarily. A timeout proves nothing. Nodes vote in quorums, retried operations must be idempotent, and protected resources must verify fencing tokens themselves. (Byzantine faults — nodes that *lie* — are a separate, harder problem you usually don't need to solve inside one org's datacenter.)

Staff engineer lens: Three review questions with high hit rates: (1) "This retry — is the operation idempotent?" (2) "This timestamp comparison — whose clock, and what happens under skew?" (3) "This distributed lock — where's the fencing token check on the storage side?" Most home-grown locking/leader-election code fails #3.

Chapter 9 — Consistency and Consensus

Chapter 8 listed everything that goes wrong; this chapter builds the abstractions that let applications survive it: **linearizability**, **ordering/causality**, and **consensus** — the same way transactions let apps pretend crashes don't happen.

Linearizability — "behave like there is only one copy"

Linearizability is the strongest common guarantee: the system behaves **as if there were a single copy of the data, and every operation happened atomically at one point in time**. Once any client reads the new value, every later read (by anyone) must also see it — no flip-flopping between old and new.

Example: Alice and Bob are refreshing for a World Cup score. Alice sees "Final: 2–1" and shouts; Bob refreshes *after* hearing her and still sees "in progress" (his read hit a lagging replica). That's a linearizability violation — the system revealed the result, then un-revealed it.

Where you truly need it: distributed locks and leader election (two leaders = split brain), uniqueness constraints (usernames, "sell only what's in stock"), cross-channel coordination (write to storage, then send a queue message referencing it). **What provides it:** single-leader replication *can* (reads from leader), consensus systems (ZooKeeper/etcd) *do*. Multi-leader and leaderless replication generally *don't* — even strict Dynamo-style quorums fail under races.

The cost: availability and latency. **CAP theorem**, stated usefully: when a network partition cuts replicas off from each other, you must choose — stay **consistent** (refuse writes on the minority side) or stay **available** (accept writes, diverge, reconcile later). And even without partitions, linearizable reads can't be served from a local replica — RAM on multi-core CPUs isn't even linearizable without memory barriers, for the same speed reasons.

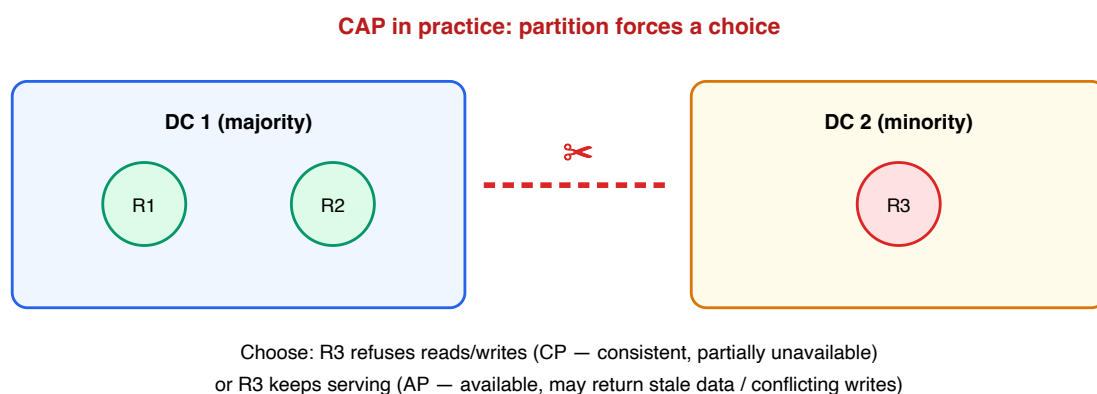


Figure 9-1. A network partition makes "consistent AND fully available" impossible; every distributed database picks a side, per operation.

Ordering and causality

Weaker than linearizability but much cheaper: **causal consistency** — anything that could have caused something else appears first, everywhere (question before answer, create-account before post). Unlike linearizability, causality only forms a *partial* order: truly concurrent events may be seen in different orders by different observers, and that's fine.

Lamport timestamps give a total order consistent with causality: each node keeps a counter, stamps events (counter, nodeID), and fast-forwards its counter whenever it sees a bigger one. But the order is only known *after the fact* — Lamport timestamps cannot tell you *right now* whether your username claim wins, because some other node might later reveal an earlier claim. For "decide now and be done," you need more...

Total order broadcast & consensus — the summit

Total order broadcast = deliver the same messages, in the same order, to all nodes, reliably. It's exactly a replication log: feed writes through it and every replica stays identical (state machine replication). A remarkable result: **total order broadcast, linearizable storage, and consensus are equivalent** — solve one and you've solved the others.

Consensus: several nodes propose values; the system decides one value; everyone agrees; the decision is final. The **FLP theorem** says this is impossible in a fully asynchronous deterministic system — but real algorithms (Paxos, **Raft**, Zab, VSR) escape by using timeouts and randomness. How they all work, in one paragraph:

- Time is divided into **epochs** (terms/ballots), each with at most one leader; epoch numbers only increase.
- Every decision needs acknowledgment from a **quorum** (usually majority) of nodes.
- Two overlapping quorums always share at least one node — so a new epoch's quorum is guaranteed to know about the previous epoch's decisions. That overlap is the entire magic: no reliance on clocks, no split brain.

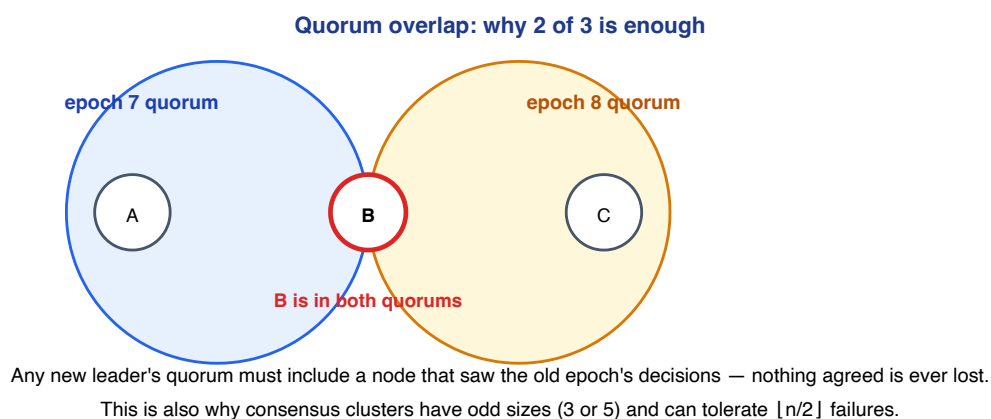


Figure 9-2. Overlapping majority quorums carry decisions across leader changes — consensus without trusting clocks.

Two-phase commit — the cautionary tale

2PC makes a transaction atomic *across* systems: a coordinator asks all participants "prepared to commit?" (phase 1); if all say yes, it decides and tells everyone (phase 2). A participant that said "yes" **must** obey the eventual decision. Fatal flaw: if the **coordinator crashes after prepare**, participants are stuck **in doubt** — holding locks, blocking other transactions, unable to commit or abort until the coordinator returns. In practice (XA transactions), coordinator outages cause orphaned locks and manual interventions. 2PC is *not* fault-tolerant consensus; real consensus algorithms avoid the single blocking coordinator.

ZooKeeper / etcd — consensus as a service

You rarely implement Raft yourself. ZooKeeper/etcd wrap consensus into an API used for: leader election, distributed locks (*with fencing tokens* — the zxid), service discovery, partition-assignment metadata. Rule of

thumb: keep **coordination** data (small, slow-changing) in ZooKeeper; keep **application** data in databases built for volume.

Key idea: Linearizability = "single copy" illusion; costly under partitions (CAP). Causal order is cheaper and often enough. When you need to *decide now* (one leader, one username winner, one commit), that's consensus — equivalent to total order broadcast — and you should get it from a proven system (ZooKeeper/etcd/Raft-based DB), never hand-roll it.

Staff engineer lens: Classify each requirement in a design: does it need linearizability (locks, uniqueness, inventory), causal ordering (feeds, threads), or mere eventual convergence (like counts)? Paying the consensus tax only where required is the difference between a fast system and a "strongly consistent" slow one. Red flag phrase to catch in reviews: "we'll just have the two services both write and they'll agree" — that's unsolved consensus.

PART III

Derived Data

Real architectures combine many datastores. This part is about **derived data** — caches, indexes, warehouses, ML features — data that is *computed from* a source of truth and can always be regenerated. Batch and stream processing are the machinery for creating and maintaining it.

Systems of record hold the authoritative version of facts; **derived data** is redundant, denormalized, and rebuildable. Being clear about which is which untangles most confusing dataflows.

Chapter 10 — Batch Processing

Three styles of system: **services** (request → response, latency-focused), **batch** (consume a big bounded input, produce an output, throughput-focused), **stream** (like batch but on unbounded, continuously arriving data). Batch processing is old (punch cards!) and deeply healthy: it's how search indexes, recommendation models, and analytics are built.

The Unix philosophy is the blueprint

`cat log | awk '{print $7}' | sort | uniq -c | sort -rn | head -5` finds your top 5 URLs from a huge log, and thanks to `sort` spilling to disk it handles data far bigger than RAM. Why this composes so well: each tool does **one thing**; a **uniform interface** (text streams) connects them; **inputs are immutable**, outputs go elsewhere — so any stage can be re-run harmlessly. MapReduce is exactly this idea across thousands of machines, with HDFS playing the role of the pipe.

MapReduce in one diagram

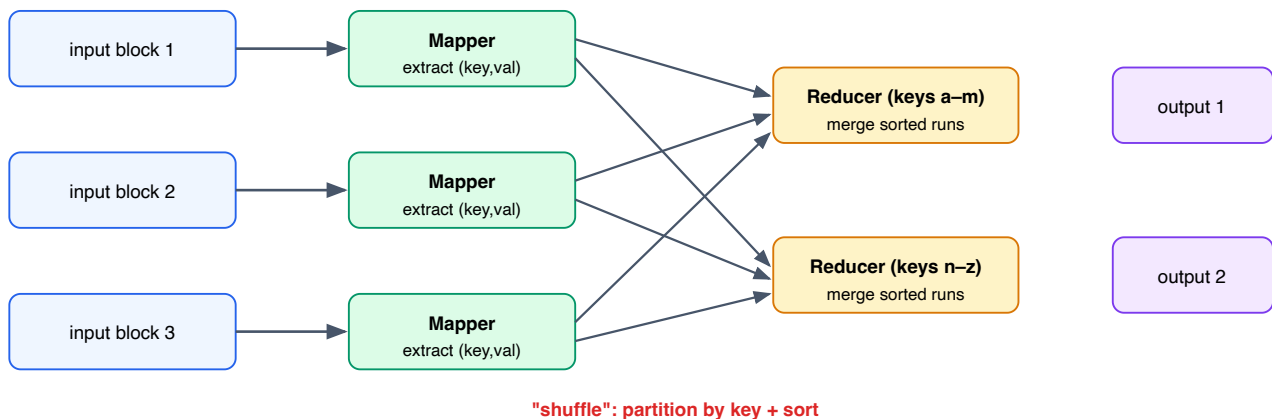


Figure 10-1. MapReduce: map extracts key-value pairs; the shuffle brings all values for a key to one reducer, sorted; reduce aggregates. Same idea as Unix `sort | uniq`, distributed.

Joins, and the skew problem

- **Reduce-side sort-merge join:** map both datasets (events + user profiles) by `user_id`; the shuffle delivers each user's events and profile to the same reducer, adjacent. No index lookups, no network chatter per record — pure sequential throughput.
- **Broadcast hash join:** if one side is small, load it into a hash table in every mapper; skip the shuffle entirely.
- **Skew:** one celebrity key can swamp a single reducer, and the whole job waits for the straggler. Fixes: spread the hot key across many reducers randomly and replicate the other side.

Why batch outputs are healthy

Inputs are **immutable**; outputs are written once, then swapped in. This buys **human fault tolerance**: deployed a bad ranking algorithm? Re-run the job on the same input — no data was destroyed (contrast with a bug that wrote garbage into a live database). Failed tasks are simply retried, since re-running a pure function

is harmless. Build databases as batch *output files* (e.g., search index shards, Voldemort read-only stores) rather than having reducers write to a live DB over the network.

Beyond MapReduce

MapReduce's weakness: every job writes all intermediate state to disk ("materialization"), and a workflow of 50 chained jobs is slow and awkward. **Dataflow engines (Spark, Flink, Tez)** model the whole workflow as one graph of operators, stream data between stages without full materialization, and recompute lost partitions from lineage on failure. Specialized layers on top: **Pregel-style graph processing** (iterative algorithms like PageRank) and high-level SQL-ish APIs that compile down to the same engines.

Key idea: Batch = pure functions over immutable inputs, retried freely, outputs swapped atomically. Joins are done by partitioning-and-sorting so the data comes to the computation. This "build derived data offline, deploy it wholesale" pattern is the most robust way to maintain indexes, models, and caches.

Staff engineer lens: Push teams away from "backfill scripts that mutate the prod DB" toward re-runnable jobs that emit fresh derived artifacts. Ask: if this pipeline ran twice, would anything break? (idempotence); if the output is wrong, can we regenerate from raw inputs? (immutability). Those two questions capture most of this chapter.

Chapter 11 — Stream Processing

Batch assumes the input is finite — but real data arrives *forever*. Stream processing is batch's unbounded sibling: process each **event** shortly after it happens instead of once a day. An event = a small, immutable record of something that happened, with a timestamp.

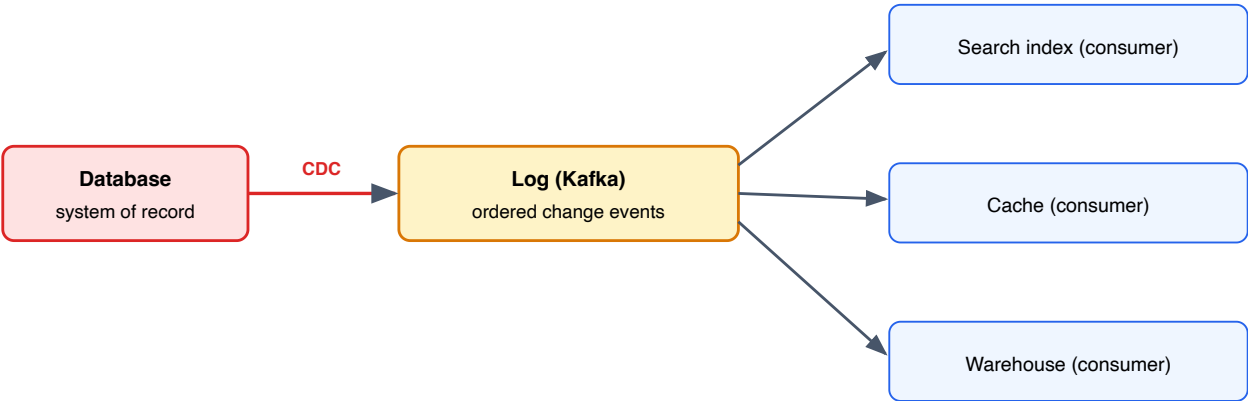
Message brokers: two families

AMQP-style (RabbitMQ)		Log-based (Kafka, Kinesis)
Model	Broker assigns messages to consumers; deletes after ack	Append-only log on disk, partitioned; consumers track an offset
Replay	Gone once acked — no replay	Re-read any past message by rewinding the offset — stream becomes more like a file
Ordering	Lost under retries/multiple consumers	Guaranteed per partition
Fits	Task queues: expensive per-message work, order irrelevant	Event streams: high throughput, order matters, many independent consumers

The log-based broker is the pivotal idea: it decouples producers from consumers (add a new consumer anytime, start from the past), and a slow consumer only falls behind its offset — it can't slow anyone else.

Change Data Capture & event sourcing — the database inside-out

CDC tails a database's replication log and publishes every insert/update/delete as a stream. Now the search index, cache, and warehouse are just **consumers of the same ordered stream** — they stay consistent with the DB without dual-write races. (Dual writes — app writes to DB *and* cache separately — are a classic bug: partial failure and reordering guarantee eventual divergence.)



All consumers apply the **same changes in the same order** → they converge. No dual-write races.

Figure 11-1. Change data capture: one ordered log feeds every derived system. Add a new consumer next year and replay from the beginning.

Event sourcing takes it further: store the *events themselves* ("student cancelled enrollment") as the source of truth, and derive current state by replaying them. Richer history, easy debugging, new views for free — at the cost of needing snapshots/compaction and handling immutability limits (GDPR deletion, mistakes). **Log compaction** (keep only the latest event per key) lets a log double as a database.

Time and windows — the hard parts

- **Event time vs processing time:** a mobile app's events may arrive *hours* after they happened (offline usage). Analyze by processing time and a restarted consumer looks like a traffic spike (like Star Wars episodes: order watched $\neq$ story order).
- **Windows** (tumbling/hopping/sliding/session) group events for aggregation. But when is a window *complete*? You can never be sure — **straggler** events arrive late. Options: ignore them (track the drop rate), or publish corrections.
- Stream joins (stream-stream, stream-table, table-table) all require keeping **state**, and results can depend on arrival order — a fact you must manage, not wish away.

Fault tolerance: exactly-once (really: effectively-once)

You can't wait for a stream to "finish" before declaring success. Techniques: **microbatching/checkpointing** (Spark Streaming, Flink) make internal state recoverable, but output side effects can still repeat. The real answers are **idempotent writes** (applying the same event twice = once) or **atomic commits of output + offset together**. "Exactly-once" always means "retries + dedup made it look like once."

Key idea: Put an ordered, replayable log at the center; make the database's changes an event stream (CDC); let every derived view (cache, index, warehouse, ML features) be a consumer that can be rebuilt by replay. Respect event-time vs processing-time, and make consumers idempotent.

Staff engineer lens: The instant-classic review catch: "you're writing to Postgres *and* Redis *and* Elasticsearch from the app — what happens when the second write fails?" Answer: dual-write inconsistency; propose CDC. Also pin teams down on late data: "what's the SLA for events arriving late, and do we drop or correct?"

Chapter 12 — The Future of Data Systems

The closing chapter is Kleppmann's opinionated synthesis: how *should* we build data systems, given everything in the book?

Data integration: unbundle the database

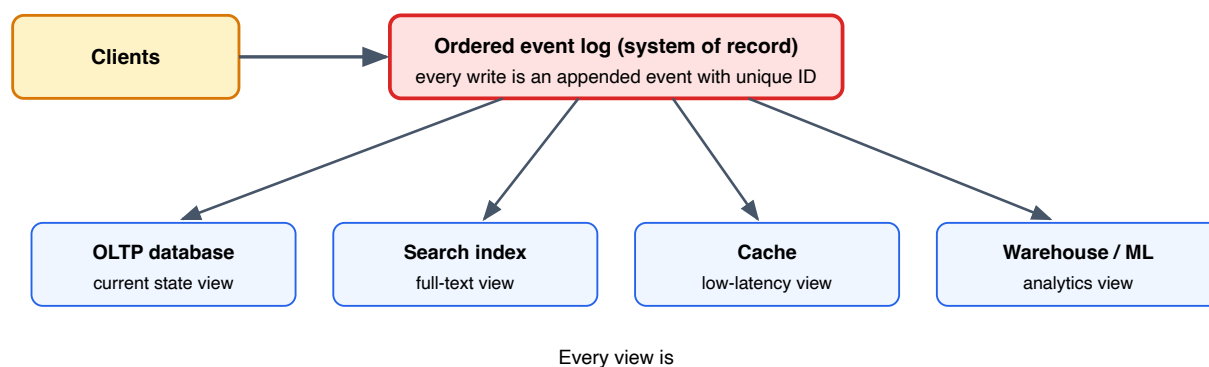
No single tool fits all access patterns, so every serious architecture is several specialized systems around shared data. The sanest way to keep them in sync is the Chapter 11 pattern generalized: **route all writes through an ordered event log; make every store a consumer**. This is "unbundling the database" — Kafka is the commit log, Elasticsearch is the secondary index, Redis is the cache, the warehouse is the materialized view; each is best-in-class, glued by dataflow. Batch and stream processors are the tools for keeping derived views fresh; **lambda architecture** (run batch and stream side by side) is the transitional form, evolving toward systems where one engine does both (replayable logs + windowed processing).

Example: "Total ordering through a log" even solves app-level races: two users try to claim username zoe . Instead of a distributed lock, both claims are appended to a partition-by-username log; the single consumer that processes that partition sees one claim first — that one wins, deterministically. Ordering via a log is often simpler than distributed transactions.

Correctness without distributed transactions

- **End-to-end idempotence:** give every operation (e.g., a payment) a client-generated unique ID that flows through the whole pipeline; every layer dedups on it. TCP's reliability doesn't survive across your load balancer, retry logic, and queue hops — only an end-to-end ID does. (The "end-to-end argument.")
- **Loosely coupled constraints:** many business rules don't need synchronous cross-partition coordination. Banks don't use distributed transactions with airlines — they use **compensating actions** (overbooked flight → apology + voucher). Enforce what must be atomic within one partition (via log ordering); handle the rest asynchronously and apologize when needed. Coordination costs latency and availability; buy it only where an apology won't do.
- **Trust, but verify (auditing):** data gets corrupted — disks rot, bugs write garbage, operators fat-finger. Don't just assume ACID means safe: **audit** — checksums, end-to-end integrity checks, comparing derived views against sources of truth. Event-sourced/immutable logs make auditing tractable: you can always recompute and compare.

The unbundled database: log-centric architecture



derived and rebuildable — schema change? Build a new view alongside, backfill by replay, switch reads over.

Figure 12-1. Writes flow through one log; every specialized store is a replaceable, replay-rebuildable materialized view.

Ethics — doing the right thing

Kleppmann ends unusually, and it matters at staff level: engineers building data systems are building society's infrastructure. Predictive analytics can automate discrimination and create feedback loops (biased training data → biased decisions → more biased data); data hoarding is a liability ("data is the pollution problem of the information age"); surveillance creeps in via defaults. Practices he urges: collect less, keep it shorter, encrypt and control access, audit algorithmic decisions for bias, treat users as "data subjects" with rights (this predates but anticipates GDPR). *We should not build systems we wouldn't want used on ourselves.*

Key idea: Architect around an ordered, replayable log of immutable events; derive every specialized view from it; enforce correctness with end-to-end IDs, partition-local ordering, compensating actions, and auditing — reserving expensive coordination for the few places an apology can't fix.

Staff engineer lens: This chapter is the staff-engineer worldview: pick per-workload tools but unify them with dataflow; ask "can we rebuild this view from raw events?" before every new datastore; ask "what's the compensating action?" before every distributed transaction; and raise data-minimization/retention in design reviews — it's an engineering concern, not just legal's.

Appendix — One-Page Cheat Sheet

Decision quick-reference

Question	Short answer
Document vs relational?	Tree-shaped, read-whole → document. Many-to-many, joins → relational. Multi-hop traversals → graph.
LSM vs B-tree?	Write-heavy, compressible → LSM (Cassandra, RocksDB). Predictable read p99 + transactional locks → B-tree (Postgres, MySQL).
OLTP vs OLAP?	Never run analyst scans on the OLTP primary. ETL/CDC into a column store.
Encoding?	Protobuf/Avro with a schema registry. Never reuse field tags. New fields optional. Test old-reader/new-writer both ways.
Replication mode?	Default single-leader. Multi-DC/offline → multi-leader + conflict plan. Max availability, no failover → leaderless quorums.
Replication lag bugs?	Read-your-writes (own data from leader), monotonic reads (pin user→replica), consistent prefix (causal order).
Partition key?	Hash for spread, range for scans; design for the hot key (compound keys, suffix-splitting). Beware scatter/gather secondary indexes.
Isolation level?	Check-then-act invariants (uniqueness, capacity, balances) need SERIALIZABLE (SSI) or explicit locks. Snapshot isolation misses write skew & phantoms.
Distributed lock?	ZooKeeper/etcd + fencing token verified by the resource . Timeout ≠ dead; clocks lie; GC pauses strike anywhere.
Need consensus?	One leader / one winner / atomic commit → yes: use etcd/ZooKeeper/Raft-based systems. Never hand-roll. 2PC blocks when the coordinator dies.
Keeping N stores in sync?	Never dual-write. CDC/event log → all consumers, idempotent, replayable.
Exactly-once?	= at-least-once + idempotence (end-to-end operation IDs) or atomic offset+output commit.

Numbers & laws worth citing

- Percentiles: report p50/p95/p99; SLOs in percentiles; the tail is amplified by fan-out (slowest backend gates the response).
- CAP (useful form): during a partition, choose consistent-but-unavailable or available-but-inconsistent — per operation, not per system.
- Quorums: n replicas, $w + r > n$ overlaps; majority quorums tolerate $\lfloor n/2 \rfloor$ node failures — hence clusters of 3 or 5.
- Failure realities: ~1 disk/day dies in a 10k-disk cluster; config error by humans is the top outage cause; GC can pause a JVM for seconds; NTP skew is tens of ms.
- Immutability buys human fault tolerance: bad code + immutable input = re-run; bad code + in-place mutation = incident.

The book in three sentences

1. Reliability, scalability, and maintainability come from choosing data models, storage engines, and encodings whose trade-offs match your workload — there is no magic database. **2.** Distributing data means replication and partitioning, whose anomalies (lag, conflicts, skew, races) are tamed by explicit guarantees — isolation levels, linearizability, causality, consensus — each with a real price. **3.** The most robust large architectures treat an ordered log of immutable events as the spine, derive every specialized view from it, and enforce end-to-end correctness with idempotence and auditing rather than blanket distributed transactions.